SYNTHAGENT: A System for Reliable LLM-Based Synthetic Data Generation in Small Data

Sancho Amaral Simões[1,*], João P. Vilela[1,2], Pedro Henriques Abreu[1]
[1]*CISUC-DEI - FCTUC, University of Coimbra*, Coimbra, Portugal
[2]*CRACS/INESCTEC-DCS - FCUP, University of Porto*, Porto, Portugal
[*]Corresponding author
sanchosimoes@student.dei.uc.pt, jvilela@fc.up.pt, pha@dei.uc.pt

***Abstract*—Many clinical prediction tasks must be learned from small data: a few hundred patient records, fewer than the task requires, that cannot be pooled with those of other hospitals. Synthetic data generation could enlarge such datasets, but generators fitted to the scarce records themselves, from interpolation methods to deep generative models, cannot add information that the records lack. Large language models (LLMs) can, because they bring knowledge from pre-training and need no training on the target data. Their raw output, however, is not a dataset. Asked directly for a fixed number of records of six medical datasets, a commercial LLM returned that number in none of 60 runs, delivering between 57% and 103% of it, and some of its responses could not be parsed. This paper presents SYNTHAGENT, a system that places a deterministic control layer between the LLM and the dataset. The control layer selects the real records the model sees, repairs and validates every response against the schema, requests exactly the missing records and logs the cost of each step, so that the dataset is completed to the requested size, for about 2% more tokens than direct prompting. We evaluate SYNTHAGENT with four commercial LLMs of two generations and two sizes and compare it with an interpolation method and two deep generative models. With the larger, more recent LLM, SYNTHAGENT yields the largest average F1 gain of all generators and never lowers the F1 of a dataset by more than 0.6 points, whereas the other generators lose up to 10.6 points. No generator improves every dataset, a limit that follows from how much information each generator can contribute.**

***Index Terms*—small data, synthetic data generation, tabular data, large language models, data augmentation, healthcare**

I. INTRODUCTION

Machine learning in medicine is usually told as a story of scale, but most clinical prediction tasks are not large. A hospital that wants to predict a complication from its own patients may have a few hundred records; a rare disease may have only a few dozen cases; and privacy rules usually prevent the hospital from combining its records with those of other institutions [1], [2]. In such settings the data are *small*: not merely few in absolute terms, but too few for the task that has to be learned from them.

What counts as too few depends on the task. Three hundred patients can suffice to predict a common outcome from a handful of variables and fall far short for a rare outcome or many variables. Our recent survey [1] makes this intuition measurable, and we follow its definition throughout the paper. Every prediction task has a *sample complexity* $n_T^\star$, the number of independent, correctly labelled records a model needs to reach a required error; for clinical models it can be estimated with the sample-size criteria of Riley et al. [3]. Every dataset, in turn, has an *effective sample size* $n_{\text{eff}}(D)$, the number of such records it is worth once repeated measurements, missing labels, label noise and differences from the target population are accounted for. A dataset is small for its task when it is worth fewer records than the task needs, that is, when the ratio $\sigma_T(D) = n_T^\star / n_{\text{eff}}(D)$, which the survey calls the *smallness index*, exceeds one; ratios up to 10 are called strained, up to 100 small, and above 100 extreme. Small data is therefore a property of the whole dataset relative to its task, not of any single class: class imbalance matters only because, for a binary outcome, $n_T^\star$ grows with the rarity of the events [3]. The six medical datasets we study illustrate the range. They have between 195 and 768 records, and their smallness indices run from 1.0 to 42.1 (Table III), so each of them holds less evidence than its task requires.

How can a model be learned from such data? The survey classifies the possible remedies by where the missing information comes from: other institutions, related tasks or pre-trained models, domain knowledge, unlabelled data, new data collection, or the transformation of the existing data, which includes synthetic data generation (SDG) [1]. SDG is attractive because it adds training records without new collection and, when the synthetic records do not reproduce real ones, can support development and testing without exposing patients [4]. The survey also sets a limit on what it can achieve. A synthetic dataset produced from D with the help of an external resource M carries no more information about the task than D and M together (Proposition 4.1 in [1]). A generator fitted only to D, such as SMOTE [5], CTGAN or TVAE [6], has no such resource: it can help a learner use D more efficiently, but it cannot supply what D does not contain.

Large language models (LLMs) are different, and this is why we turn to them. An LLM is not trained on D. It receives a few real records in its prompt and writes new ones using what it learned during pre-training, for example that `age` is a positive integer that tends to rise with blood pressure [7], [8]. It is, in the terms of the survey, a generator that brings its own M. The difficulty is that an LLM response is a piece of text, not a dataset. Figure 1 shows what can go wrong when a model is asked for 30 records: it may return fewer, add a feature that does not exist, omit one that does, or write a

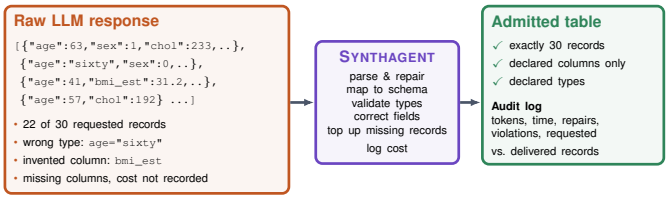


Fig. 1. The problem addressed in this paper, in an illustrative example. Asked for 30 records of a heart-disease dataset, an LLM response (left) can contain too few records, invented or missing features and values of the wrong type. SYNTHAGENT (right) validates and corrects every response, so the resulting dataset has the requested size and schema, and a log records each correction and its cost.

TABLE I
REQUIREMENTS MET BY EXISTING GENERATOR FAMILIES AND BY SYNTHAGENT. ✓: BY DESIGN; (✓): PARTIALLY (SMOTE'S SIZE FOLLOWS THE CLASS BALANCE; FINE-TUNED LMS DISCARD UNPARSABLE RECORDS); ×: NOT ADDRESSED.

Family	No training on D	R1: exact size	R2: schema validated	R3: cost measured
Interpolation [5]	✓	(✓)	×	×
Deep generators [6], [9], [10]	×	✓	×	×
Fine-tuned LMs [7], [11]	×	✓	(✓)	×
Prompted LLMs [8], [12]	✓	×	×	×
SYNTHAGENT (ours)	✓	✓	✓	✓

number as a word. Our experiments confirm that this is not a rare event. When we asked a commercial LLM directly for a fixed number of records, it returned exactly that number in none of 60 runs, delivering on average 87.6% of it, and ten of its responses were not valid JSON at all (Section VI-A). Each request also costs tokens and time that are rarely reported. An LLM, in short, brings the right kind of information in an unreliable form.

This is the gap the paper addresses, and it can be stated as a set of requirements. Given a small dataset D, its schema S (the features and their types) and a requested number of synthetic records N, we want a synthetic dataset G, produced without training a generator on D, that has exactly N records (R1), conforms to S (R2) and comes with a measured cost (R3), and that is also faithful to D (R4), does not reproduce its records (R5) and improves, or at least does not degrade, the classifiers trained on D (R6). The first three requirements describe a dependable data product and can be guaranteed by a system; the last three describe a useful one and can only be measured. Existing generators address different subsets of them (Table I). SMOTE needs no training but only interpolates between existing records and ignores feature types. Deep generators [6], [9], [10] and language models fine-tuned on the target data [7], [11] must be trained on the very records that are scarce. LLMs used through prompting [8], [12] avoid training, but the work on them has studied what to put in the prompt and which generated records to keep; whether the output is a well-formed dataset of the requested size, and at what cost, has been left to the model.

We present SYNTHAGENT, a system built on one idea: the LLM proposes records, and a deterministic *control layer* decides which of them enter the dataset. The control layer surrounds every request to the model. It keeps an explicit record of the schema, the real records shown, the records accepted so far and the number still missing, and it consists of four components, each of which removes one of the failures in Figure 1: a *context selector* chooses the real records that fit in the prompt; a *parser* repairs malformed output and maps the returned features to the schema; a *validator* checks feature types and asks the model to correct only the invalid values; and a *completion loop* requests exactly the missing records until N is reached. Every token, correction and violation is logged. With these components, SYNTHAGENT satisfies R1–R3 by construction and without training on D.

Whether the resulting records are also useful is the question the rest of the paper answers. Our objective is to determine whether a control layer makes LLM-based SDG a reliable and useful option for small data, and four research questions lead to the answer, each building on the previous one. We first ask whether the control layer delivers what direct prompting does not, and which of its components change the result beyond the variation between repeated runs (RQ1, reliability). If the records arrive as requested, we ask whether they are faithful to the real data without reproducing it (RQ2, quality), and then whether they improve classifiers trained on small data, given how much those classifiers already vary between data splits (RQ3, utility). Finally, we ask how SYNTHAGENT compares with established generators and how much its results depend on the LLM behind it (RQ4, comparison).

The paper makes three contributions. The first is SYNTHAGENT itself, an open-source system that makes LLM-based SDG for small data satisfy R1–R3 by construction (Section IV). The second is an evaluation protocol for SDG in small data, which pairs every classifier trained with synthetic records with the same classifier trained on real records alone, on the same split, and which measures how much two runs of the same pipeline differ before attributing any difference to a design choice (Section V). The third is an empirical study on six medical datasets with four LLMs, three established generators and a direct-prompting baseline, comprising 840 dataset–configuration runs and up to 25,200 classifier fits (Section VI).

The study supports three findings. First, the control layer is what turns LLM output into a dataset: direct prompting never delivered the requested size, whereas SYNTHAGENT did so by construction, with 99.7% to 100% of the values filled, for 1.9% more tokens, and without changing fidelity or utility. Second, with SYNTHAGENT, LLM-based SDG is the most robust augmentation in F1 among the generators compared: with the larger, more recent LLM it yields the largest average F1 gain (+1.09 points) and never lowers the F1 of a dataset by more than 0.6 points, while SMOTE, CTGAN and TVAE lose up to 8.2, 10.6 and 2.9 points. Third, in small data the variation between repeated runs is as large as the effect of most design choices; only the choice of LLM exceeds it, and no generator improves utility significantly on every dataset, as

the survey's conservation result anticipates.

The remainder of the paper follows the same path. Section II places SYNTHAGENT among existing generators and identifies the limitations it addresses. Section III formalizes the problem, and Section IV describes how the control layer meets R1–R3. Section V explains how R4–R6 are measured, Section VI answers the research questions, and Section VII discusses what the answers imply.

II. Background and Related Work

To see where SYNTHAGENT fits, we first review the families of tabular SDG methods, then examine what each family offers when data are small, and finally consider the work on validated output and on the evaluation of synthetic data on which our design and protocol build.

A. Synthetic Data Generation for Tabular Data

Tabular SDG methods fall into four families. *Interpolation and statistical models* include SMOTE, which creates minority-class records on the segment between two neighbours [5], and Bayesian networks, which have also been used for differentially private release [13]. *Deep generative models* learn the joint distribution with a neural network: CTGAN and TVAE [6], diffusion models such as TabDDPM, CoDi and TabSyn [9], [10], [14], and relational and cross-silo variants [15], [16]. *Fine-tuned language models* serialize each record as text and fine-tune a language model on the target data, as in GReaT [7], REaLTabFormer [11] and TabLoft [17]. *Prompted LLMs* generate records from examples placed in the prompt, without training, as in CLLM [8] and EPIC [12].

B. Synthetic Data Generation in Small Data

Most of these generators were designed and benchmarked on datasets with tens or hundreds of thousands of records, such as Adult (48,842 records) or Covertype (581,012 records) [6], and their behaviour on a few hundred records has received far less attention. Each family meets small data with a different limitation, and naming the limitations makes clear which ones a new system must address. Interpolation methods and deep generators are fitted only to D; by the conservation result of [1] they cannot add information about the task, and they ignore what the features mean (L1, no external information). Deep generators and fine-tuned language models must also estimate many parameters from few records, so they can produce records outside the observed ranges and types, or reproduce training records [18] (L2, training on scarce records). Prompted LLMs avoid both problems, but nothing ensures that their output is a valid dataset of the requested size, and its cost is not measured (L3, unreliable output); CLLM [8] filters generated records by their learning dynamics and EPIC [12] designs balanced prompts, but both concern the quality of the records that are kept rather than R1–R3. Across all families, finally, generators are usually compared from single runs, so differences caused by run-to-run variation, which is large in small data, are reported as effects (L4, evaluation without a noise estimate). SYNTHAGENT addresses L3 with its control layer and L4 with its evaluation protocol, and, because it uses a pre-trained LLM, it inherits the advantages of prompted LLMs over L1 and L2. The next subsection explains where the idea of a control layer comes from.

C. LLM Agents, Validated Output and Data Quality

The control layer follows the agent pattern of alternating actions and observations [19], [20], but its observations are deterministic checks rather than the model's own reflections. This choice connects SYNTHAGENT to a long line of data-management work. Synthetic data have long been used to test systems and share data [15], [21]; Kamino enforces denial constraints during private synthesis [22]; data-cleaning systems combine statistical inference with integrity constraints [23], [24]; foundation models can clean and integrate data through prompting [25]; and recovering structured records from free-form LLM output is a data-engineering problem in its own right [26]. SYNTHAGENT applies the same principle to generation: the schema and type rules are inferred from the data, and every response is parsed and validated against them. Unlike feature augmentation, which adds columns from related tables [27], it adds records.

D. Evaluating Synthetic Data

Data-preparation studies have shown that a step that looks beneficial can harm downstream models [28], [29], and evaluation frameworks such as Synthcity and SynthEval recommend assessing fidelity, privacy and utility together [4], [30]. Fidelity can be complemented by a classifier two-sample test, which asks whether a classifier can tell real from synthetic records [31], and privacy by membership-inference attacks, which ask whether an attacker can tell which records were used for generation [18], [32]. Formal guarantees require syntactic anonymization models such as ℓ-diversity and t-closeness [33], [34] or differential privacy [35], which SYNTHAGENT does not claim. Our protocol (Section V) adopts the multi-dimensional view and adds the paired comparison and noise estimate that small data requires.

III. Problem Formulation

The problem statement of Section I mixes two kinds of requirement, and making the distinction explicit shapes both the system and its evaluation. Let D^{tr} and D^{te} be a training and a test partition of a small dataset D with schema S, and let a configuration θ of the system (LLM, context strategy, active rules) produce a synthetic dataset G_θ from D^{tr} without access to D^{te}. Requirements R1 and R2 define the feasible set $\mathcal{F}(S, R, N)$ of datasets with exactly N records that conform to S and to the active rules R. The other requirements are objectives: a fidelity loss $L_{\mathrm{fid}}(D^{\mathrm{tr}}, G_\theta)$ for R4, a privacy risk $L_{\mathrm{priv}}(D^{\mathrm{tr}}, G_\theta)$ for R5, the change in downstream utility $\Delta U(G_\theta)$ for R6 and the generation cost $\kappa(\theta)$, which R3 requires to be measured and which should also be kept low:

$$\theta^\star \in \operatorname*{arg\,min}_{\theta:\, G_\theta \in \mathcal{F}(S,R,N)} \big(L_{\mathrm{fid}},\ L_{\mathrm{priv}},\ -\Delta U,\ \kappa\big)(\theta). \qquad (1)$$

The objectives in Equation (1) can conflict: records closer to the real ones are more faithful but easier to attack, and a larger LLM may produce better records at a higher cost. We therefore do not collapse them into one weighted score, because the right weights depend on the use; a hospital sharing data for software testing weights privacy and validity, whereas a team augmenting a classifier weights utility. The formulation also divides the work of the paper. The constraint $G_\theta \in \mathcal{F}(S, R, N)$ is the responsibility of the system, which the next section describes; the objectives are the subject of the evaluation that follows it.

IV. The Synthagent System

Synthagent meets the constraint of Equation (1) by never trusting the LLM with it. Figure 2 shows the architecture. A profiling step infers the schema S and the type rules R from the training data. Each request then passes through four stages: the context selector builds the prompt (1), the LLM proposes a batch of records (2), the parser and validator check it (3), and the accepted records are counted against the requested size (4). Each type of failure triggers its own correction. We describe first the state that ties these stages together and then each stage in turn.

A. State of the Control Layer

Let N be the requested number of synthetic records and R the active rules. At step t, the control layer keeps the state

$$z_t = (S,\, R,\, C_t,\, A_t,\, N - |A_t|,\, B_t,\, E_t), \tag{2}$$

where C_t is the set of real records shown to the LLM, A_t the accepted synthetic records, $N - |A_t|$ the number still missing, B_t the remaining token budget and E_t the log of parser and validator results. The LLM, denoted π_ϕ, proposes a batch $Y_t \sim \pi_\phi(\cdot \mid z_t)$. Let P_S be the parser, which also maps the returned features to S, $V_{S,R}$ the validator and K a correction step that sends invalid values back to the LLM, at most twice. The accepted set is updated as

$$A_{t+1} = A_t \cup K_{V_{S,R}}\big(P_S(Y_t)\big). \tag{3}$$

Equation (3) expresses the central design decision: the LLM only proposes, and a record enters A_{t+1} only after it has been parsed, mapped to S and validated. Because the state is explicit rather than implied by the conversation history, the number of missing records is always known, and E_t records every failure for later inspection. The remaining subsections describe how each component reads and updates this state.

B. Context Selection (R1, R3)

The context selector decides which real records the LLM sees. Table II lists exactly what the model receives in our experiments. The core evidence is a 20% sample of the real training split, serialized as compact row-major JSON with the ordered feature columns and the observed target, so the model sees how features and class co-occur in real records. No free-text description of the clinical task is added, so that no dataset receives more prompt engineering than another; the model must infer the role of each feature from its name and values.

Formally, a selection strategy $s \in \{\text{random}, \text{dense}, \text{diverse}\}$ defines an ordered candidate pool, and the token budget determines how much of that pool is visible:

$$\begin{aligned} C_t &= \mathrm{Slice}_t\big(\mathrm{Select}_s(D^{\mathrm{tr}}, \lceil 0.2|D^{\mathrm{tr}}|\rceil), k_t\big)\,, \\ k_t &= \max\{k : T(H, I, C_{t,k}, O_t) \le 0.85W\}. \end{aligned} \tag{4}$$

Here H is the schema and output contract, I optional instruction text, O_t the reserved output, W the backend context window and T a token estimator (2.8 characters per token for compact JSON). The slice operator rotates through the pool across chunks, so successive requests see different patients when the pool exceeds one window. If the provider still rejects a request as too long (HTTP 400 or 422), the control layer shrinks the visible context by 40% and retries, at most three times. "20% context" therefore describes the candidate pool, not a promise that every candidate row appears in every prompt.

The 20% fraction and the 85% window limit were fixed before the experiments and not tuned. The default strategy samples the context at random. *Dense* prefers records with the most observed cells, reducing ambiguity from missing fields. *Diverse* standardizes numerical features, runs k-means and selects the records nearest to the centroids, so that the context covers more of the feature space. Neither strategy is class-stratified, so neither guarantees that a rare diagnosis appears in context; Section VII returns to this limitation.

C. Parsing, Validation and Correction (R2)

The parser and the validator decide which parts of a response can enter the accepted set A_t of Equation (3). Records are requested in JSON batches of at most 30, a size that keeps every response within the output limit of the smallest LLM tested and limits the loss when a response fails. A local parser first extracts structured content and fixes common defects (non-finite literals, trailing commas, concatenated objects). If parsing still fails, the control layer issues one repair request at temperature 0.1 that quotes the error position and surrounding text. The returned feature names are then mapped back to the schema.

Profiling infers a type rule for every feature, and optional boundary rules (numerical ranges with 10% tolerance, or categorical domains with at most 20 values). Dynamic Pydantic models validate each batch. The validator groups failures by field and reports one representative bad value per field, so the correction prompt does not grow linearly with repeated errors. Up to two correction rounds are attempted per batch. User-defined post-processing rules (noise, rounding, normalization) can be applied deterministically afterwards.

In the reference configuration of our experiments, type rules are active, boundary rules are off and no user rules are supplied, so we evaluate the default system rather than the strictest one. Rules are used only by the validator and are never inserted into the prompt.

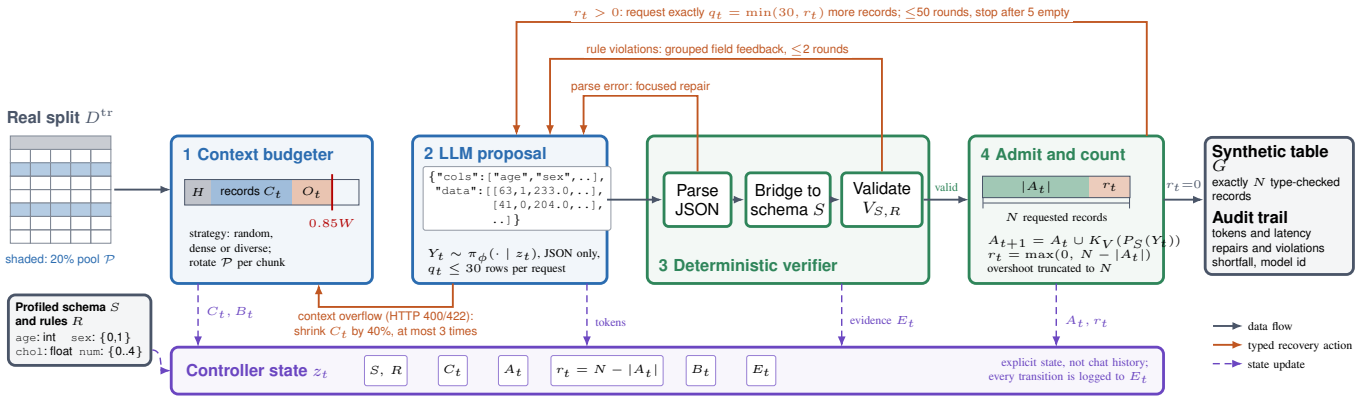


Fig. 2. Architecture of SYNTHAGENT. Real records enter a context window of limited size (1); the LLM proposes a JSON batch (2); deterministic checks parse it, map it to the schema and validate the type rules (3); and accepted records are counted against the requested size (4). Each failure type triggers its own correction (orange). Every stage reads or updates the state z_t of the control layer (purple), which also serves as the audit log of the run.

TABLE II
INFORMATION BOUNDARY FOR ONE GENERATION REQUEST IN THE REPORTED STUDY. "NOT SUPPLIED" IS INTENTIONAL: THOSE SIGNALS CANNOT INFLUENCE THE GENERATED RECORDS.

Context layer	What the model receives	Why it is included or excluded
Schema	Ordered column names and an exact column-count requirement	Fixes the record interface and prevents invented or omitted fields.
Demonstrations	Compact row-major serialization of a 20% sample of the training records, with feature and target values	Exposes joint patterns, value formats, label prevalence and plausible cross-column combinations without sending the whole local table.
Generation task	Exact number of new records for the current batch, at most 30	Turns an open-ended completion into a bounded action whose output count can be verified.
Output contract	JSON only, fixed column order, complete cells, raw numeric values, no extra metadata	Makes parsing deterministic and removes interface failures unrelated to statistical quality.
Dynamic feedback	Parser error location after malformed JSON, or grouped field violations after failed validation	Reveals only the evidence needed for a targeted repair instead of replaying the full history.
Free-text metadata	Not supplied	Prevents unequal, dataset-specific prompt engineering from confounding the comparison.
Test information	No holdout rows, predictions, metrics or classifier identities	Preserves the evaluation boundary and prevents optimization against the test set.
Global statistics	No separately computed means, quantiles, correlations or class quotas	Fidelity must arise from the selected records; the consequence is weaker protection against rare-mode omission.

D. Completion Loop (R1)

LLMs often return fewer records than requested, as RQ1 will show. The completion loop derives its next request from the accepted state rather than from the conversation:

$$r_t = \max(0, N - |A_t|), \qquad q_t = \min(30, r_t). \tag{5}$$

Equation (5) requests only the residual q_t, and any overshoot is truncated. The loop runs for at most 50 top-up rounds and stops after five consecutive empty or unparsable responses. Successful runs therefore return exactly N records; persistent provider failure can still end a run short, and that shortfall is recorded rather than hidden. Algorithm 1 summarizes the complete loop.

The last element of the control layer is its audit log, which serves R3. Every run records prompt, completion and total tokens, LLM generation time, parser repair attempts, rule violations, the requested and delivered record counts and the model identifier. These records make a synthetic dataset inspectable after the fact and give the cost term $\kappa(\theta)$ in Equation (1) a measured value.

E. Deployment Across Hospitals

The setting that motivates this work, small datasets held by hospitals that cannot pool them [1], [2], [36], also suggests how SYNTHAGENT can be deployed. Because the control layer needs only a local dataset, its schema and an inference endpoint, each hospital of a network can run it inside its own infrastructure. Consider K hospitals, each holding a small dataset D_k for the same prediction task, with different case mix, coding practice and class prevalence. Each hospital runs SYNTHAGENT locally and trains on its real and synthetic records,

$$G_k = \mathcal{A}(D_k, S_k, R_k, B_k), \qquad \Delta w_k = \mathcal{U}(w_t, D_k \cup G_k), \tag{6}$$

Algorithm 1 Generation loop of the control layer

Require: training records D^{tr}, schema S, rules R, size N, strategy s, window W

$\mathcal{P} \leftarrow \mathrm{Select}_s(D^{\mathrm{tr}}, \lceil 0.2|D^{\mathrm{tr}}| \rceil)$; $A \leftarrow \emptyset$; $E \leftarrow [\,]$
for chunk $t = 1, \ldots, \lceil N/30 \rceil$ **do**
  $C_t \leftarrow$ rotated slice of $\mathcal{P}$ that fits $0.85W$ ▷ Eq. (4)
  $Y \leftarrow \pi_\phi(S, C_t, \min(30, N - 30(t-1)))$; on overflow shrink C_t by 40%, retry ≤ 3
  $X \leftarrow P_S(Y)$; **if** parse fails **then** $X \leftarrow P_S(\mathrm{repair}(Y))$
  for $j = 1, 2$ **do** ▷ targeted correction
    if $V_{S,R}(X)$ has no violations **then break**
    $X \leftarrow P_S(\pi_\phi(\text{grouped violations of } X))$; append to E
  end for
  $A \leftarrow A \cup X$
end for
$u \leftarrow 0$
while $|A| < N$ **and** fewer than 50 rounds **and** $u < 5$ **do** ▷ Eq. (5)
  $X \leftarrow P_S(\pi_\phi(S, \text{first 20 records of } \mathcal{P}, \min(30, N - |A|)))$
  if $X = \emptyset$ **then** $u \leftarrow u + 1$ **else** $A \leftarrow A \cup X$; $u \leftarrow 0$
end while
return first N rows of A, together with E, token counts and timings

where S_k and R_k are the local schema and rules, B_k the local token budget and $\mathcal{U}$ the local training procedure. Only the model update Δw_k and approved aggregate diagnostics leave the hospital. A coordinator aggregates the updates as in FedAvg [37], [38],

$$w_{t+1} = w_t + \sum_{k \in \mathcal{K}_t} \frac{n_k}{\sum_{j \in \mathcal{K}_t} n_j} \Delta w_k, \qquad n_k = |D_k|, \quad (7)$$

preferably under secure aggregation [39]. The weights count *real* records, $|D_k|$ rather than $|D_k \cup G_k|$, so a hospital gains no extra weight by generating more records (Figure 3). Unlike SiloFuse, which trains a shared generator across sites [16], this design needs no generator training across hospitals, and each hospital can reject its synthetic records when paired local validation shows no gain. The design is not, by itself, a privacy guarantee: with a remote LLM the prompt content leaves the hospital, so sensitive deployments need a locally served model or a data-processing agreement, and both synthetic records and model updates can leak information [40]. Our experiments evaluate the generation step one hospital at a time, which is the setting the next section describes; a multi-site evaluation is future work.

V. Experimental Setup

The system guarantees R1–R3 by construction; R4–R6 must be measured, and in small data measuring them reliably is itself a challenge. This section describes the data, the protocol and the measures with which we answer RQ1–RQ4, and explains why each choice was made.

A. Datasets

We use healthcare data because small data is most common there and because the case for SDG, more training records without exposing patients, is strongest [1]. We use public datasets so that every result can be reproduced. Table III lists the six classification datasets: five from the UCI repository [41]–[45] and the Pima Indians diabetes benchmark [46]. They are widely used, they are all small for their tasks ($\sigma > 1$), and together they span the factors that make small data harder: 195 to 768 records, 8 to 30 features, two to five classes, a smallest class from 4.3% to 37.3% of the records, and smallness indices across the strained and small regimes. We keep the original five-class heart-disease target and impute the few missing values (0.1% of the cells in two datasets) with the mean for numerical and the mode for categorical features.

TABLE III
Evaluation datasets. n: records; d: features; k: classes; $n_{\min}$: records in the smallest class; $n^\star$: sample complexity by the first criterion of Riley et al. [3] (shrinkage 0.9, $d(k-1)$ parameters, anticipated R^2_{CS} of 15% of its maximum); $\sigma = n^\star / n_{\mathrm{eff}}$ with n_{eff} the training size ($0.8n$).

Dataset	n	d	k	$n_{\min}$ (%)	$n^\star$	σ
Heart disease	303	13	5	13 (4.3)	10,201	42.1
Breast cancer	569	30	2	212 (37.3)	2,302	5.1
Heart failure	299	12	2	96 (32.1)	946	4.0
Diabetes	768	8	2	268 (34.9)	621	1.0
Liver disease	583	10	2	167 (28.6)	808	1.7
Parkinson's	195	22	2	48 (24.6)	1,851	11.9

For heart disease, $n^\star$ applies the binary criterion to the smallest class versus the rest and is an approximation. Parkinson's contains several recordings per subject, so its true n_{eff} is lower and its σ higher than shown.

B. Protocol

Measuring whether synthetic records help a classifier requires real records that the generator has never seen, so each dataset is split into a training partition, given to the generator, and a test partition, used only to score classifiers. A single split, however, is unreliable in small data: the test partition holds only 39 to 154 records, and the result changes with the split. We therefore repeat the split ten times with seeds 43 to 52 (80/20 holdouts) and report means and standard deviations. In each repetition the generator receives the training partition and produces as many synthetic records as there are training records; the LLM sees a 20% sample of the training partition as context and generates at temperature 0.3, a value fixed in advance to favour adherence to the output format while keeping some diversity.

The utility of synthetic data also depends on the learner: interpolated records can help a nearest-neighbour classifier and harm a boosted ensemble. To avoid conclusions tied to one learner, we train ten classifiers that cover the main model families: linear (logistic regression, LR), kernel (support vector machine, SVM), distance-based (k-nearest neighbours, kNN), probabilistic (Gaussian naive Bayes, NB), a single tree (decision tree, DT), bagging (random forest, RF), boosting (gradient boosting, GB; AdaBoost; XGBoost) and a neural network (multilayer perceptron, MLP). Each classifier is trained in three regimes and scored on the same test partition: real records only, synthetic records only, and real plus synthetic records. XGBoost failed on liver disease in every repetition because the labels are coded $\{1, 2\}$; the code now re-encodes

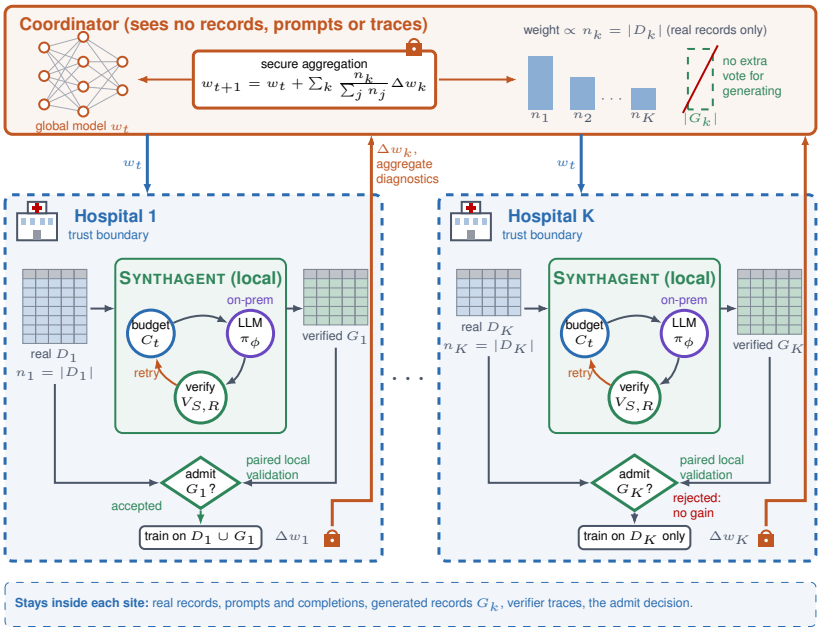


Fig. 3. Deployment across hospitals. Each hospital runs SYNTHAGENT (context selection, a locally served LLM, validation and correction) inside its own infrastructure, then decides with paired local validation whether to use the synthetic records G_k (Hospital 1) or to train on real data only (Hospital K). Only the model update Δw_k and approved aggregate diagnostics leave the hospital. The coordinator weights each update by the number of real records $n_k = |D_k|$ (Equation (7)), so generating more records gives a hospital no extra weight.

them, but filling this cell would require regenerating every configuration, so we report 59 of 60 classifier–dataset cells.

C. Configurations

We evaluate 14 configurations and analyse only those that completed all six datasets and ten repetitions. The first group answers RQ1. A *direct-prompting* baseline uses the same prompts and context as SYNTHAGENT but none of its control layer: the response is parsed with a strict JSON parser, and there is no repair, schema mapping, correction, retry or completion loop. Seven *control-layer variants* then change one element of the pipeline at a time: the reference SYNTHAGENT; three variants that disable a component (type rules, post-processing, and a legacy context-join option); dense and diverse context selection; and a variant in which the LLM chooses context size, selection strategy and temperature from a profile of the dataset. All eight use GPT-4o mini, the least expensive LLM, so that every comparison runs at the lowest cost.

The second group answers RQ4. Four *LLMs* run the reference configuration: GPT-4o mini, GPT-4.1 mini, GPT-4o and GPT-4.1, a 2×2 design that crosses model generation (4o versus 4.1) with model size (mini versus full). Drawing them from one provider keeps the interface, tokenizer and output mode constant, so that differences can be attributed to the model; the control layer itself works with any OpenAI-compatible endpoint, including locally served open models. Three *established generators* use the same splits: SMOTE [5], which balances every class to the majority count and therefore emits only non-majority records, and CTGAN and TVAE [6] with the default settings of the reference implementation (300 epochs). In total, the 14 configurations give 840 dataset–configuration runs and up to 25,200 classifier fits.

Three of the control-layer variants turn out to change nothing in this setting. With no user rules, disabling post-processing removes a step that does nothing; the context-join option affects only a legacy fidelity score; and disabling type rules cannot change the output because the validator never found a violation with GPT-4o mini. Together with the reference, these variants are therefore four independent runs of the same pipeline, and we use them as such: their spread estimates the variation between repeated runs, which we call the *noise floor*. A difference between two configurations that is smaller than this spread is not evidence of an effect.

D. Measures

Each requirement of the problem statement has its own measure. For *reliability* (R1–R3), we record the number of records delivered against the number requested, the number of malformed responses, completeness (the percentage of non-empty values), the corrections made by the control layer and the cost in tokens and generation time. For *fidelity* (R4), we compare the synthetic records alone, without the real context, with the real training records over the numerical features $\mathcal{J}$:

$$F_{\mathrm{JSD}} = 100\Big(1 - \tfrac{1}{|\mathcal{J}|}\textstyle\sum_{j\in\mathcal{J}} D_{\mathrm{JS}}(D_j, G_j)\Big),$$
$$F_{\mathrm{corr}} = 100\big(1 - \mathrm{mean}\,|\Sigma_D - \Sigma_G|\big), \tag{8}$$

where D_{JS} is the Jensen–Shannon divergence between real and synthetic marginals and Σ_D, Σ_G are their correlation matrices; both scores equal 100 for identical distributions. We add a classifier two-sample test (C2ST) [31], in which a random forest is trained to separate real from synthetic records and an accuracy close to 0.5 means they are hard to tell apart, and, for the established generators and direct prompting, the percentage of *valid* records, whose values lie in the observed ranges and categories and are integers where the real feature is, a stricter criterion than R2, which concerns features and types only. For *privacy* (R5), a membership-inference attack scores each real record by its distance to the nearest synthetic record and

reports the ROC AUC between training and test records, where 0.5 is chance. For *utility* (R6), the paired change for metric $m \in \{\text{accuracy}, \text{F1}\}$, dataset d, classifier c and repetition h is

$$\Delta m_{d,c,h} = m\big(D^{\text{tr}}_{d,h} \cup G_{d,h};\, c\big) - m\big(D^{\text{tr}}_{d,h};\, c\big), \qquad (9)$$

with both models scored on the same test partition, so that variation between test partitions is not mistaken for an effect of SDG. We use binary F1 for the binary tasks and class-weighted F1 for heart disease; binary F1 scores the class coded 1, which is the minority class for heart failure and diabetes and the majority class for liver disease and Parkinson's. Changes are reported in percentage points (pp).

E. Statistical Analysis

Repetitions and classifiers are not independent datasets, so we average them within each dataset and treat the six datasets as matched blocks. Configurations are compared with the Friedman test,

$$\chi^2_F = \frac{12M}{Q(Q+1)} \sum_{a=1}^{Q} \bar{\text{rk}}_a^{\,2} - 3M(Q+1), \qquad (10)$$

where $\bar{\text{rk}}_a$ is the average rank of configuration a over $M = 6$ datasets and Q the number of configurations, followed by two-sided Wilcoxon signed-rank tests of the best-ranked configuration against every other, with Holm correction [47], [48]. Per-cell tests of augmented versus real-only training over the ten repetitions are Holm-corrected within each metric. With six blocks the tests have limited power, which is precisely why we report effect sizes and the noise floor alongside p-values. With these measures in place, we can now answer the research questions in order.

VI. Results

A. RQ1: What the Control Layer Delivers

The first question is whether the control layer is needed at all, and the direct-prompting baseline answers it. Table IV compares it with the reference SYNTHAGENT, both with GPT-4o mini, on the same prompts, context and splits. Direct prompting returned the requested number of records in none of the 60 runs. It fell short in 50 runs, by 4% to 43%, and overshot in the 10 diabetes runs, by up to 3.4%. On average it delivered 87.6% of the requested records, and as little as 64.0% on the 30-feature breast-cancer dataset, where the model often returned fewer than the 30 records requested per batch. Ten responses, eight on breast cancer and two on Parkinson's, were not valid JSON and lost an entire batch each. SYNTHAGENT delivers exactly the requested number by construction, because its completion loop requests the missing records and truncates any excess, and its parser recovers malformed responses instead of discarding them.

The control layer achieves this without changing the quality of the records. The two configurations have the same marginal fidelity (JSD 72.7), similar correlation preservation (71.2 versus 72.6) and C2ST accuracy (0.883 versus 0.878), a membership AUC near chance (0.517 versus 0.526) and

TABLE IV
DIRECT PROMPTING VERSUS SYNTHAGENT (RQ1), BOTH WITH GPT-4O MINI, OVER TEN SPLITS. DELIVERED: RECORDS RETURNED AS A PERCENTAGE OF THOSE REQUESTED (RANGE OVER SPLITS); MALF.: MALFORMED RESPONSES; ΔF1: MEAN F1 CHANGE (PP).

		Direct prompting			SYNTHAGENT	
Dataset	N	Delivered (%)	Malf.	ΔF1	Delivered	ΔF1
Heart disease	242	93.5 (91.7–94.6)	0	−0.15	100	−0.97
Breast cancer	455	64.0 (56.9–69.0)	8	−0.78	100	−0.62
Heart failure	239	94.5 (93.3–95.8)	0	+0.26	100	−1.00
Diabetes	614	102.2 (101.3–103.4)	0	+2.23	100	+2.39
Liver disease	466	93.8 (92.7–95.5)	0	+0.12	100	+0.13
Parkinson's	156	77.3 (62.2–87.8)	2	−1.94	100	−0.77
Mean		87.6	10	−0.04	100	−0.14

mean utility changes within the noise floor estimated below (-1.36 versus -1.19 pp in accuracy, -0.04 versus -0.14 pp in F1). The direct responses contained no type errors or unknown features with this LLM, so its failures were failures of volume and syntax, exactly the ones the completion loop and the parser address. The cost of the control layer is small: 73,510 tokens per repetition against 72,126, or 1.9% more. The two configurations differ, in other words, not in what the records look like but in whether the user receives the dataset that was requested.

Having established that the control layer is needed, we ask which of its design choices change the outcome. Table V compares the seven control-layer variants. The first four rows run the same pipeline, so their spread is the noise floor: JSD 72.43–72.92, correlation 70.83–72.93, mean accuracy change -1.80 to -1.19 pp, mean F1 change -0.18 to $+0.28$ pp and attack AUC 0.523–0.534. Differences of about 0.5 JSD points, 2 correlation points or 0.6 pp of utility between two runs are therefore not evidence of a design effect. The same rows show that generation time is a poor cost measure: the four equivalent runs take 228 to 379 seconds per repetition, depending on the provider, while their token counts differ by less than 2%. We therefore measure cost in tokens.

Against this floor, the design choices have small effects. Dense context stays inside the floor on every measure. Diverse context is the only choice that leaves it on fidelity, with the highest JSD (73.27) and correlation (73.72) and the lowest C2ST accuracy (0.872), but also the highest attack AUC (0.544): records that cover more of the feature space lie closer to more real records. Letting the LLM choose its own settings stays inside the floor on fidelity and utility and costs 57% more tokens (115,245 versus 73,510). The Friedman tests find no difference in utility between the variants ($\chi^2_F(7) = 3.10$, $p = .875$ for accuracy; $\chi^2_F(7) = 4.39$, $p = .734$ for F1), and every Holm-adjusted comparison has $p = 1.0$. The ranks in Figure 4 show how little the variants separate: LLM-chosen settings rank best on accuracy (3.75), and the no-post-processing variant, which changes nothing, ranks best on F1 (3.33). SMOTE ranks sixth of eight on accuracy (5.00) and second on F1 (3.67).

The correction steps were rarely needed with GPT-4o mini:

TABLE V

CONTROL-LAYER VARIANTS (RQ1), ALL WITH GPT-4O MINI. THE FIRST FOUR ROWS RUN THE SAME PIPELINE AND DEFINE THE NOISE FLOOR. SECONDS AND TOKENS ARE MEANS PER REPETITION. BOLD: BEST VALUE AMONG THE LLM CONFIGURATIONS; LOWER IS BETTER FOR C2ST, FOR THE DISTANCE OF THE ATTACK AUC FROM 0.5, AND FOR COST.

	Fidelity (R4) and privacy (R5)				Utility change (R6, pp)		Cost per repetition (R3)	
Configuration	JSD	Corr.	C2ST	Attack AUC	Acc.	F1	Seconds	Tokens
Noise floor: four runs of the same pipeline								
Reference SYNTHAGENT	72.70	72.57	0.878	0.526	−1.19	−0.14	378.7	73,510
No type rules	72.92	72.93	0.876	0.534	−1.80	−0.04	321.6	**72,188**
No post-processing	72.43	70.83	0.883	**0.523**	−1.71	+0.28	251.5	72,235
No context join	72.52	71.18	0.880	0.531	−1.31	−0.18	**228.1**	72,576
Design choices								
Dense context	72.87	70.88	0.878	0.525	−1.24	+0.07	238.8	72,600
Diverse context	**73.27**	**73.72**	**0.872**	0.544	−1.34	−0.13	271.1	73,757
LLM-chosen settings	72.53	70.10	0.886	0.526	−1.25	+0.11	310.2	115,245

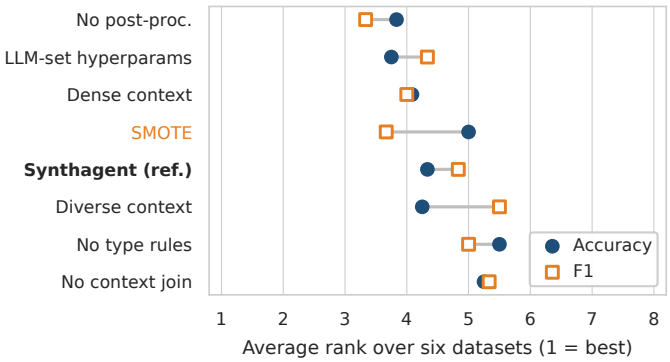


Fig. 4. Average ranks of the control-layer variants and SMOTE over the six datasets (lower is better). Neither accuracy nor F1 shows a significant difference.

it produced no type violation, and the local parser fixed every defect in its responses without a repair request. They were needed with GPT-4.1 mini, the fastest LLM we tested, which produced on average 7.4 type violations per repetition on breast cancer and one malformed response in ten repetitions on diabetes. These were detected and sent back for correction instead of entering the dataset, and the resulting datasets were complete and gave the best mean accuracy change of all configurations (Table VIII). A faster but less disciplined model can thus be used safely, because its failures are caught. Across all LLMs, the synthetic datasets have 99.7% (GPT-4o mini) to 100.0% of their values filled.

Answer to RQ1. The control layer is what makes LLM output usable as a dataset: direct prompting never returned the requested size and lost whole batches to malformed responses, whereas SYNTHAGENT delivers complete datasets of the requested size, records its cost and catches the failures of less disciplined models, for 1.9% more tokens. Within one LLM, no design choice moves fidelity or utility beyond the noise floor, except a small fidelity gain from diverse context. The value of the control layer lies in reliability rather than in a higher score, which leads to the next question: how good are the records it delivers?

TABLE VI

REFERENCE CONFIGURATION (GPT-4O MINI) PER DATASET, AVERAGED OVER TEN REPETITIONS AND ALL CLASSIFIERS (RQ2, RQ3). JSD AND CORRELATION ARE ON A 0–100 SCALE.

	R4 and R5				Acc. (%)		Change (pp)	
Dataset	JSD	Corr.	C2ST	AUC	Real	Aug.	Acc.	F1
Breast	77.2	74.5	0.911	0.550	95.1	94.4	−0.66	−0.62
Diabetes	80.2	44.0	0.897	0.504	74.2	74.1	−0.03	+2.39
Heart dis.	65.2	76.6	0.893	0.507	56.6	53.4	−3.15	−0.97
Heart fail.	71.2	81.2	0.892	0.534	74.1	71.6	−2.53	−1.00
Liver	71.8	73.8	0.883	0.495	67.4	67.6	+0.19	+0.13
Parkinson's	70.6	85.3	0.789	0.567	84.9	84.0	−0.97	−0.77
Mean	72.7	72.6	0.878	0.526	75.4	74.2	−1.19	−0.14

B. RQ2: Fidelity and Privacy

Table VI reports the reference configuration per dataset. Marginal fidelity averages 72.7 out of 100, from 65.2 on the five-class heart-disease dataset, the smallest relative to its task ($\sigma = 42.1$), to 80.2 on diabetes, the closest to sufficient ($\sigma = 1.0$). The membership-inference attack stays near chance, at 0.526 on average and at most 0.567 on any dataset, so the synthetic records do not reveal which real records were used; this is a diagnostic for one attack, not a formal guarantee. The C2ST accuracy of 0.878 shows that a classifier can still separate real from synthetic records, so we do not claim that the synthetic records can replace real ones. *Answer to RQ2.* The synthetic records follow the real marginals reasonably well without reproducing the real records, but they remain distinguishable from real data. Whether records of this quality help a classifier is the next question.

C. RQ3: Utility for Classifiers Trained on Small Data

Any utility effect must be judged against how much classifiers trained on real data alone already vary, so we measure that first (Table VII). The variation is large. On the 195-record Parkinson's dataset, MLP accuracy is $72.3 \pm 18.2\%$, while tree ensembles exceed 92%. On heart failure, SVM and kNN reach 63.3% and 61.3% accuracy but an F1 of only 0.0 and 0.7,

because they almost never predict the minority class; accuracy alone would hide the very failure that SDG is meant to address.

Against this background, the effect of synthetic records depends on both the dataset and the classifier (Table VI, Figure 5). With the reference configuration, accuracy improves in 18 of 59 classifier–dataset cells, decreases in 38 and is unchanged in three; F1 improves in 26, decreases in 31 and is unchanged in two. The largest gains occur where real-only training fails on the minority class: kNN on heart failure, with a real-only F1 of 0.7%, gains 15.3 F1 points, and MLP on Parkinson's, the most unstable baseline, gains 8.2 accuracy points. Classifiers that are already strong lose slightly. After Holm correction over 59 tests per metric, no single cell is significant at 0.05. Figure 6 shows the variation between splits: the accuracy change is consistently negative on the two heart datasets and centred on zero for liver disease, and diabetes is the only dataset with a clearly positive F1 change at unchanged accuracy.

Answer to RQ3. With GPT-4o mini, synthetic records help the classifiers that fail on the minority class and slightly harm the strong ones; on average they leave F1 unchanged (-0.14 pp) and lower accuracy (-1.19 pp). This is what the conservation result of [1] leads one to expect: synthetic records can add only the information their generator brings, so large average gains are unlikely. If the LLM is the source of that information, a stronger LLM should matter, and so should the choice between an LLM and a generator fitted to the data. RQ4 examines both.

D. RQ4: Comparison with Established Generators and LLMs

The choice of LLM is the only factor whose effect exceeds the noise floor (Table VIII, Figure 7). All three alternatives to GPT-4o mini exceed it in accuracy change and two in F1 change. GPT-4.1 gives the highest marginal fidelity (75.72, 2.8 points above the floor) and the largest F1 gain ($+1.09$ pp); GPT-4.1 mini gives the best accuracy change (-0.23 pp) at the lowest generation time. The three stronger LLMs preserve pairwise correlations less well (62.8–64.3, against at least 70.83 for GPT-4o mini), so better marginals come at the cost of weaker dependencies. Across six datasets these differences are not significant ($\chi^2_F(3) = 2.00$, $p = .572$ for accuracy; $\chi^2_F(3) = 1.80$, $p = .615$ for F1); the closest comparison, GPT-4.1 mini against GPT-4o mini in accuracy, differs by $+0.86$ pp with $p_{\mathrm{Holm}} = .188$.

How does LLM-based synthesis compare with generators fitted to the data? Table IX places SYNTHAGENT with GPT-4o mini and GPT-4.1 next to SMOTE, CTGAN and TVAE. No generator is best on every measure. CTGAN reaches the highest marginal fidelity (JSD 77.6), but its records are the easiest to detect (C2ST 0.936) and only 29.6% of them are valid (1.0% on Parkinson's). TVAE keeps 75.6% valid records and SMOTE 85.7%, although only 14.4% on heart disease, where interpolation produces fractional values in integer-coded features. All five generators keep the membership attack near chance (0.503–0.532). Cost separates them sharply: SMOTE needs 0.015 seconds, TVAE and CTGAN 5.6 and 14.6 seconds of CPU time, and SYNTHAGENT 73,000 to 81,000 tokens and several minutes of API time, with no training.

Average changes, however, hide how much each generator can harm an individual dataset, and in small data a large loss on one dataset matters more than a small gain on average. Table X reports the F1 change per dataset. SMOTE gives the largest gains where the scored class is rare ($+10.91$ pp on heart failure, $+4.11$ on diabetes) but loses 8.20 pp on liver disease, where the scored class is the majority. CTGAN loses on every dataset, by up to 10.65 pp. TVAE gives the best accuracy change on average but loses 2.86 pp of F1 on heart failure. SYNTHAGENT with GPT-4.1 has both the largest mean F1 gain ($+1.09$ pp) and the smallest worst case: it never lowers the F1 of a dataset by more than 0.53 pp, while raising it by 6.35 pp on heart failure. Across datasets, the differences between the five generators are not significant ($\chi^2_F(4) = 6.80$, $p = .147$ for accuracy; $\chi^2_F(4) = 5.60$, $p = .231$ for F1; every Holm-adjusted $p \geq .25$); TVAE has the best accuracy rank (2.00), and the two SYNTHAGENT configurations share the best F1 rank (2.50).

Answer to RQ4. The LLM is the main lever on quality and utility. With GPT-4.1, SYNTHAGENT gives the largest and most consistent F1 gain of the five generators, at a higher cost than the established generators and without a significant advantage across six datasets. TVAE remains a cheap option for accuracy, and SMOTE for a known rare positive class.

VII. DISCUSSION AND LIMITATIONS

We set out to determine whether a control layer makes LLM-based SDG a reliable and useful option for small data, and the four answers combine into a clear picture. Reliability is achieved, and it is achieved by the control layer: the same LLM, prompted directly, never returned the dataset that was requested, whereas SYNTHAGENT completes it by construction, records its cost and caught the failures of a less disciplined model, at almost no extra cost (RQ1). Usefulness is real but bounded: the synthetic records are faithful without reproducing real ones, yet distinguishable from them (RQ2); they help classifiers that fail on the minority class (RQ3); and with a stronger LLM they give the most consistent F1 gain among the generators compared (RQ4). No generator, however, improves utility significantly on all datasets. This is what our survey's conservation result predicts [1]: SDG cannot replace missing patients, and its benefit is bounded by the knowledge the generator brings. Among the generators we compared, the LLM is the only one that brings such knowledge, and the control layer is what makes it usable.

These results translate into four practical recommendations. Choose the LLM first, since it is the only factor whose effect exceeds the variation between runs. Measure cost in tokens rather than time, which depends on the provider. Check the effect of synthetic records per classifier, on paired splits, before using them. And do not let the LLM choose its own settings without measuring the extra cost.

The conclusions are subject to several limitations. Only six datasets act as independent blocks, which limits the power

TABLE VII

REAL-ONLY BASELINES (RQ3) OVER TEN 80/20 SPLITS: MEAN IN PERCENT, STANDARD DEVIATION IN SMALL TYPE. XGBOOST ON LIVER DISEASE (–) FAILED IN EVERY SPLIT BECAUSE THE LABELS ARE CODED $\{1, 2\}$.

	Heart dis.		Breast		Heart fail.		Diabetes		Liver		Parkinson's	
Classifier	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
RF	61.0 ±4.4	57.0 ±5.9	95.6 ±1.8	95.0 ±2.8	84.3 ±3.3	76.8 ±4.4	74.9 ±2.8	60.4 ±6.0	71.1 ±3.5	81.4 ±2.4	92.3 ±3.2	94.8 ±2.2
DT	53.4 ±3.9	54.0 ±5.0	93.3 ±2.3	92.1 ±4.0	79.2 ±4.7	68.4 ±9.2	71.1 ±3.8	57.2 ±6.4	66.0 ±3.2	76.3 ±3.1	85.1 ±5.6	89.7 ±3.8
MLP	56.6 ±6.4	52.5 ±7.6	94.7 ±0.9	94.0 ±2.1	48.3 ±12.9	34.2 ±24.7	68.4 ±4.1	52.9 ±12.0	68.4 ±4.1	79.4 ±3.6	72.3 ±18.2	77.8 ±26.1
SVM	54.8 ±6.8	39.0 ±7.9	93.3 ±2.1	92.4 ±3.4	63.3 ±5.2	0.0 ±0.0	75.6 ±2.2	57.3 ±4.7	70.7 ±2.9	82.8 ±2.0	81.0 ±8.2	88.5 ±5.2
LR	59.8 ±5.6	56.5 ±5.0	95.2 ±2.0	94.6 ±2.6	80.5 ±3.6	68.9 ±8.4	77.3 ±2.0	62.8 ±5.4	70.3 ±4.0	81.1 ±3.2	86.9 ±5.6	91.3 ±3.9
kNN	55.4 ±6.3	44.4 ±7.9	94.3 ±1.8	93.4 ±3.2	61.3 ±4.1	0.7 ±2.1	75.5 ±1.8	60.0 ±3.3	68.6 ±4.2	79.7 ±3.2	83.6 ±5.6	89.5 ±3.9
NB	55.7 ±5.3	56.9 ±5.0	93.9 ±1.7	93.1 ±3.2	75.0 ±4.9	56.7 ±13.1	76.4 ±2.4	64.0 ±4.9	55.9 ±3.7	55.9 ±3.8	71.5 ±4.4	76.6 ±4.0
GB	54.1 ±5.5	52.8 ±6.3	95.9 ±2.2	95.4 ±2.8	83.0 ±4.9	75.4 ±7.2	74.7 ±3.0	61.7 ±5.8	67.9 ±3.3	78.7 ±2.4	92.1 ±4.0	94.7 ±2.6
AdaBoost	60.8 ±7.6	60.7 ±7.3	97.1 ±1.5	96.8 ±1.7	82.5 ±3.0	74.2 ±6.5	74.9 ±2.2	60.8 ±4.7	67.4 ±3.4	78.2 ±3.3	91.3 ±3.8	94.2 ±2.5
XGBoost	54.3 ±4.6	52.0 ±5.1	97.2 ±1.4	96.9 ±1.8	83.5 ±4.0	75.5 ±7.0	72.7 ±2.8	59.1 ±6.2	–	–	93.3 ±5.2	95.6 ±3.2

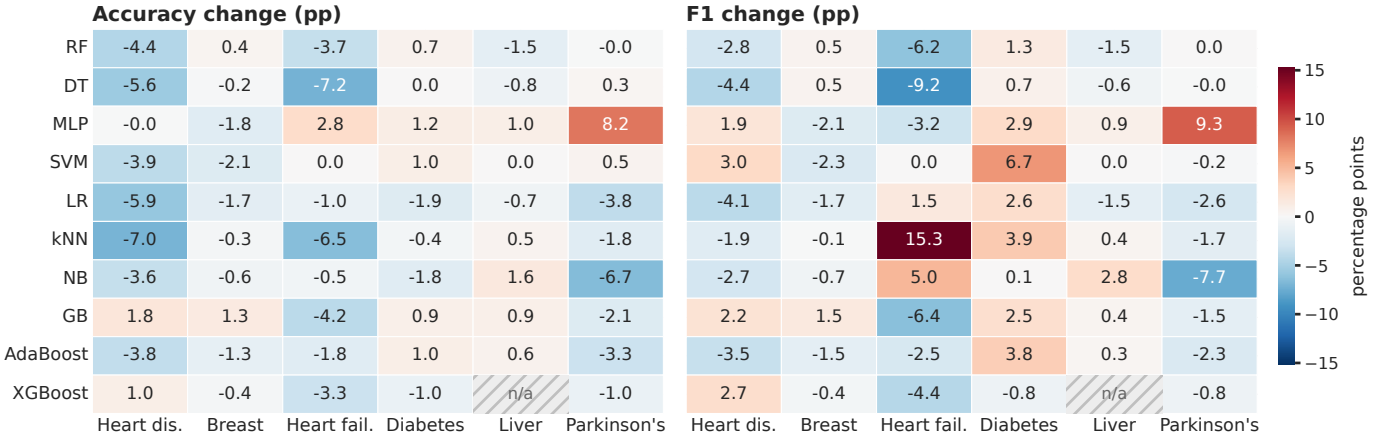


Fig. 5. Change from real-only to real-plus-synthetic training for the reference configuration (RQ3), per classifier and dataset. Hatched cells (n/a) were not evaluated. The same synthetic data can help one classifier and harm another.

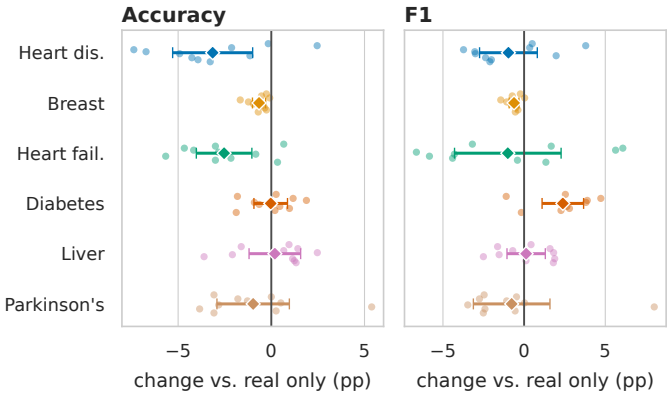


Fig. 6. Variation between splits for the reference configuration (RQ3). Points are splits, after averaging the paired changes over classifiers; diamonds are means and bars 95% t intervals.

of the tests, and the splits are not stratified. The Parkinson's dataset contains several recordings per subject without a subject identifier, so recordings of one person can appear in both partitions. Schema and value ranges are inferred once from the full dataset, which exposes test extremes to the profiler; train-only profiling would be stricter. JSD uses fixed

TABLE VIII

LLMS UNDER THE REFERENCE CONFIGURATION (RQ4). SECONDS AND TOKENS ARE MEANS PER REPETITION.

LLM	JSD	Corr.	ΔAcc.	ΔF1	Sec.	Tokens
GPT-4o mini	72.70	**72.57**	−1.19	−0.14	378.7	**73,510**
GPT-4.1 mini	73.60	62.82	−0.23	+0.92	**168.5**	72,069
GPT-4o	73.95	64.32	−0.50	+0.25	255.2	80,046
GPT-4.1	**75.72**	63.30	−0.44	+1.09	201.7	80,652

histograms and is dominated by numerical features, and the membership attack is a single distance-based adversary, so no result establishes differential privacy [35]. The SYNTHAGENT runs predate the counter of delivered records, so their size follows from the completion loop of Algorithm 1 rather than from a logged count, and the validity measure was logged only for the established generators and direct prompting (78.6% valid records), so it is not compared with SYNTHAGENT. The correction paths were exercised only by GPT-4.1 mini; a stress test with weaker or locally served models and stricter rules is needed to measure how often each one is needed, and in the current implementation the output of the last correction round and the records obtained in top-up rounds are not validated

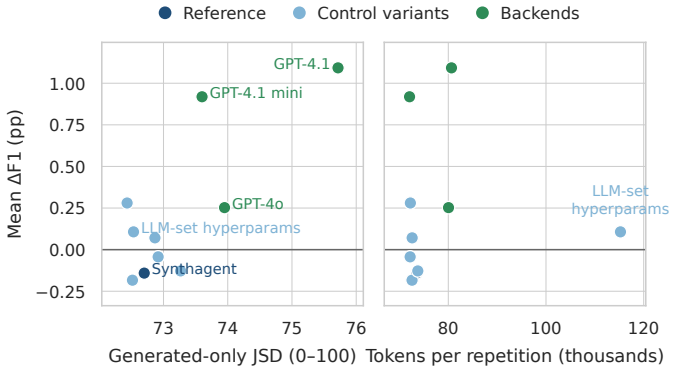


Fig. 7. Fidelity, F1 change and token cost of the LLM configurations (RQ1, RQ4). Control-layer variants cluster inside the noise floor; the LLMs spread along fidelity and F1.

TABLE IX
GENERATORS COMPARED ON THE SAME SPLITS (RQ4). VALID: % OF RECORDS WITHIN THE OBSERVED RANGES, CATEGORIES AND INTEGER TYPES (NOT LOGGED FOR THE LLM RUNS, WHOSE TYPES ARE ENFORCED BY THE VALIDATOR). SECONDS: API TIME FOR SYNTHAGENT, CPU TIME OTHERWISE.

Generator	JSD	Corr.	C2ST	AUC	Valid	ΔAcc.	ΔF1	Sec.	Tok.
SYNTHAGENT, 4o mini	72.7	72.6	0.878	0.526	–	−1.19	−0.14	379	73.5k
SYNTHAGENT, 4.1	75.7	63.3	0.877	0.527	–	−0.44	+1.09	202	80.7k
SMOTE†	69.9	85.0	0.795	0.532	**85.7**	−3.08	+0.33	**0.01**	0
CTGAN	**77.6**	75.2	0.936	**0.503**	29.6	−1.40	−3.37	14.6	0
TVAE	71.9	**85.6**	0.868	0.515	75.6	−0.09	−0.21	5.6	0

†Emits only non-majority records, so its C2ST is not comparable.

again. Context selection is not class-stratified, so a rare class can be missing from a prompt. All LLMs come from one provider; GPT-4.1 nano and o4-mini were started but did not complete every dataset and are excluded. CTGAN and TVAE use default settings, and tuned or diffusion-based generators may do better on some datasets. Finally, the smallness indices in Table III rely on a default anticipated R^2_{CS} and are estimates.

Synthetic medical records can support development, teaching and software testing, but they must not inform diagnosis or policy without domain review, and near-chance performance against one membership attack does not justify releasing sensitive data. To support verification, the implementation, the experiment runner and the script that regenerates every table and figure from the stored results are publicly available.[1]

VIII. CONCLUSION

Small clinical datasets pose a dilemma for synthetic data generation. Generators fitted to the available records cannot add the information those records lack, whereas an LLM can bring knowledge from pre-training but returns text rather than a dataset. SYNTHAGENT resolves the dilemma with a control layer that turns LLM responses into complete, validated datasets of the requested size and records their cost, without training on the target data. On six small medical datasets,

[1]TODO(authors): add the public repository URL required by the ICDE artifact policy.

TABLE X
MEAN F1 CHANGE PER DATASET (PP) OVER TEN SPLITS AND ALL CLASSIFIERS (RQ4). SA: SYNTHAGENT. WORST: LARGEST LOSS ON ANY DATASET.

Generator	Heart d.	Breast	Heart f.	Diab.	Liver	Park.	Mean	Worst
SA, 4o mini	−0.97	−0.62	−1.00	+2.39	+0.13	−0.77	−0.14	−1.00
SA, 4.1	−0.44	−0.07	+6.35	+1.75	−0.50	−0.53	+1.09	−0.53
SMOTE	−1.50	−0.53	+10.91	+4.11	−8.20	−2.83	+0.33	−8.20
CTGAN	−2.30	−2.00	−10.65	−4.22	−0.65	−0.38	−3.37	−10.65
TVAE	−0.73	−0.88	−2.86	+0.89	+2.21	+0.10	−0.21	−2.86

direct prompting never delivered the requested dataset, while SYNTHAGENT completes it by construction, at 1.9% more tokens; with the larger, more recent LLM it gave the largest and most consistent F1 gain of five generators, although no generator was significantly better on every dataset. Because repeated runs of the same pipeline vary as much as most design choices, we recommend that SDG for small data be evaluated against such a noise floor. Future work will evaluate locally served LLMs, class-aware context selection and the deployment across hospitals outlined in Section IV-E.

ACKNOWLEDGMENT

This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 – Centre for Informatics and Systems of the University of Coimbra, https://doi.org/10.54499/UID/00326/2025.

REFERENCES

[1] S. A. Simões, J. P. Vilela, and P. H. Abreu, "Collectively big, locally small: Formalizing small data in healthcare and learning privately across siloed hospitals," 2026, manuscript under review.

[2] N. Rieke, J. Hancox, W. Li, F. Milletar'i, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein, S. Ourselin, M. Sheller, R. M. Summers, A. Trask, D. Xu, M. Baust, and M. J. Cardoso, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, no. 1, p. 119, 2020.

[3] R. D. Riley, K. I. E. Snell, J. Ensor, D. L. Burke, F. E. Harrell, K. G. M. Moons, and G. S. Collins, "Minimum sample size for developing a multivariable prediction model: Part II – binary and time-to-event outcomes," *Statistics in Medicine*, vol. 38, no. 7, pp. 1276–1296, 2019.

[4] Z. Qian, R. Davis, and M. van der Schaar, "Synthcity: A benchmark framework for diverse use cases of tabular synthetic data," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[6] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[7] V. Borisov, K. Sessler, T. Leemann, M. Pawelczyk, and G. Kasneci, "Language models are realistic tabular data generators," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=cEygmQNOeI

[8] N. Seedat, N. Huynh, B. van Breugel, and M. van der Schaar, "Curated LLM: Synergy of LLMs and data curation for tabular augmentation in low-data regimes," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024.

[9] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 17 564–17 579.

[10] H. Zhang, J. Zhang, Z. Shen, B. Srinivasan, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=4Ay23yeuz0

[11] A. V. Solatorio and O. Dupriez, "REaLTabFormer: Generating realistic relational and tabular data using transformers," *arXiv preprint arXiv:2302.02041*, 2023.

[12] J. Kim, T. Kim, and J. Choo, "EPIC: Effective prompting for imbalanced-class data synthesis in tabular data classification via large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[13] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2014.

[14] C. Lee, J. Kim, and N. Park, "CoDi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 18 940–18 956.

[15] J. Fan, T. Liu, G. Li, J. Chen, Y. Shen, and X. Du, "Relational data synthesis using generative adversarial networks: A design space exploration," *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1962–1975, 2020.

[16] A. Shankar, H. Brouwer, R. Hai, and L. Y. Chen, "SiloFuse: Cross-silo synthetic data generation with latent tabular diffusion models," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 110–123.

[17] L. Chen, C. Wu, J. Li, S. Liu, G. Ye, and H. Chai, "TabLoft: Tabular data generation based on LLM with ordered features," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 1044–1056.

[18] J. Hyeong, J. Kim, N. Park, and S. Jajodia, "An empirical study on the membership inference attack against tabular data synthesis models," *arXiv preprint arXiv:2208.08114*, 2022.

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[20] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[21] A. Sanghi and J. R. Haritsa, "Synthetic data generation for enterprise DBMS," in *Proceedings of the IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 3585–3588.

[22] C. Ge, S. Mohapatra, X. He, and I. F. Ilyas, "Kamino: Constraint-aware differentially private data synthesis," *Proceedings of the VLDB Endowment*, vol. 14, no. 10, pp. 1886–1899, 2021.

[23] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *Proceedings of the IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 458–469.

[24] J. Qin, S. Huang, Y. Wang, J. Zhu, Y. Zhang, Y. Miao, R. Mao, M. Onizuka, and C. Xiao, "BClean: A Bayesian data cleaning system," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 3407–3420.

[25] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[26] J. Rorseth, P. Godfrey, L. Golab, D. Srivastava, and J. Szlichta, "Recovering structure in unstructured LLM outputs," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 4329–4330.

[27] J. Liang, C. Lei, X. Qin, J. Zhang, A. Katsifodimos, C. Faloutsos, and H. Rangwala, "Featpilot: Automatic feature augmentation on tabular data," in *Proceedings of the IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 2148–2160.

[28] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A study for evaluating the impact of data cleaning on ML classification tasks," in *Proceedings of the IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 13–24.

[29] S. Guha, F. A. Khan, J. Stoyanovich, and S. Schelter, "Automated data cleaning can hurt fairness in machine learning-based decision making," in *Proceedings of the IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 3747–3754.

[30] A. D. Lautrup, T. Hyrup, A. Zimek, and P. Schneider-Kamp, "SynthEval: A framework for detailed utility and privacy evaluation of tabular synthetic data," *Data Mining and Knowledge Discovery*, vol. 39, no. 1, p. 6, 2025.

[31] D. Lopez-Paz and M. Oquab, "Revisiting classifier two-sample tests," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=SJkXfE5xx

[32] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy*, 2017, pp. 3–18.

[33] A. Machanavajjhala, J. Gehrke, D. Kifer, and M. Venkitasubramaniam, "ℓ-diversity: Privacy beyond k-anonymity," in *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*, 2006, p. 24.

[34] N. Li, T. Li, and S. Venkatasubramanian, "t-closeness: Privacy beyond k-anonymity and ℓ-diversity," in *Proceedings of the IEEE 23rd International Conference on Data Engineering (ICDE)*, 2007, pp. 106–115.

[35] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9, no. 3–4.

[36] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: Facilitating multi-institutional collaborations without sharing patient data," *Scientific Reports*, vol. 10, p. 12598, 2020.

[37] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Ag"uera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[38] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[39] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.

[40] Q. Chen, Y. Chen, B. Yan, X. Jiang, X. Zhang, Y. Kang, T. Zhang, W. Huang, C. Gao, L. Fan, and Q. Yang, "Model trip: Enhancing privacy and fairness in model fusion across multi-federations for trustworthy global healthcare," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024.

[41] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart disease," UCI Machine Learning Repository, 1988.

[42] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast cancer wisconsin (diagnostic)," UCI Machine Learning Repository, 1993.

[43] "Heart failure clinical records," UCI Machine Learning Repository, 2020.

[44] B. V. Ramana, M. S. P. Babu, and N. B. Venkateswarlu, "ILPD (indian liver patient dataset)," UCI Machine Learning Repository, 2012.

[45] M. Little, "Parkinsons," UCI Machine Learning Repository, 2007.

[46] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 1988, pp. 261–265.

[47] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[48] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.